

Practical 11

17ce049

Aim - Classification using Support Vector Machine on IRIS dataset

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
```

In [2]:

```
# import some data to play with
iris = datasets.load_iris()
X = iris.data[:, :2] # we only take the first two features. We could
X1=iris.data[:,2:5]
# avoid this ugly slicing by using a two-dim dataset
y = iris.target
x_train,x_test,y_train,y_test = train_test_split(X,y,random_state=0,test_size=0.25)
x_train1,x_test1,y_train1,y_test1 = train_test_split(X1,y,random_state=0,test_size=0.25)
```

In [3]:

```
print(X1.shape)
print (X.shape)
```

```
(150, 2)
(150, 2)
```

In [4]:

```
# we create an instance of SVM and fit out data. We do not scale our
# data since we want to plot the support vectors
# SVM regularization parameter C = 1.0
# Kernel can be {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'}, default='rbf'

svc_rbf = svm.SVC(kernel='rbf', C=1,gamma=1).fit(x_train, y_train)
svc_poly= svm.SVC(kernel='poly', C=1,gamma=1).fit(x_train, y_train)
svc_linear=svm.SVC(kernel="linear",C=1,gamma=1).fit(x_train,y_train)
```

In [5]:

```
svc_rbf1 = svm.SVC(kernel='rbf', C=1,gamma=1).fit(x_train1, y_train1)
svc_poly1= svm.SVC(kernel='poly', C=1,gamma=1).fit(x_train1, y_train1)
svc_linear1=svm.SVC(kernel="linear",C=1,gamma=1).fit(x_train1, y_train1)
```

In [6]:

```

predict_rbf = svc_rbf.predict(x_test)
predict_rbf1 = svc_rbf1.predict(x_test1)
predict_poly = svc_poly.predict(x_test)
predict_poly1 = svc_poly1.predict(x_test1)
predict_linear = svc_linear.predict(x_test)
predict_linear1 = svc_linear1.predict(x_test1)

```

In [15]:

```

print("Accuracy for rbf with first two feature "+ str(accuracy_score(y_test,predict_rbf)*100))
print("Accuracy for rbf with 3 and 4 feature "+ str(accuracy_score(y_test1,predict_rbf1)*100))

print("Accuracy for poly with first two feature "+ str(accuracy_score(y_test,predict_poly)*100))
print("Accuracy for poly with 3 and 4 feature "+ str(accuracy_score(y_test1,predict_poly1)*100))

print("Accuracy for linear with first two feature "+ str(accuracy_score(y_test,predict_linear)*100))
print("Accuracy for linear with 3 and 4 feature "+ str(accuracy_score(y_test1,predict_linear1)*100))

print ("\nHence the accuracy is good with linear regularization in accordance to the first

```

```

Accuracy for rbf with first two feature 73.68421052631578
Accuracy for rbf with 3 and 4 feature 97.36842105263158
Accuracy for poly with first two feature 73.68421052631578
Accuracy for poly with 3 and 4 feature 97.36842105263158
Accuracy for linear with first two feature 76.31578947368422
Accuracy for linear with 3 and 4 feature 97.36842105263158

```

Hence the accuracy is good with linear regularization in accordance to the first two features and accuracy is higher when we train model on 3rd and 4th feature.

In [28]:

```

# create a mesh to plot in
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
h = (x_max / x_min)/100
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
np.arange(y_min, y_max, h))

```

In [29]:

```

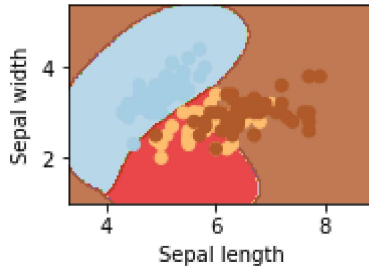
#create a mesh for the plot for feature 3 and 4 we find that the minimum is 0 so the i =inf.
x_mi, x_ma = X1[:, 0].min() - 1, X1[:, 0].max() + 1
y_mi, y_ma = X1[:, 1].min() - 1, X1[:, 1].max() + 1
i = 0.02
xx1, yy1 = np.meshgrid(np.arange(x_mi, x_ma, i), np.arange(y_mi, y_ma, i))

```

In [30]:

```
plt.subplot(2, 2, 1)
Z = svc_rbf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx.min(), xx.max())
plt.title('SVC with rbf kernel for first two feature')
plt.show()
```

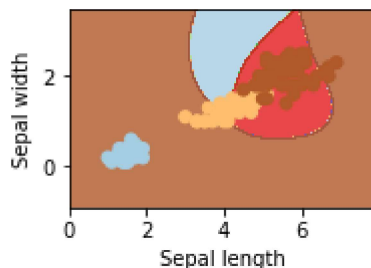
SVC with rbf kernel for first two feature



In [12]:

```
plt.subplot(2, 2, 1)
Z1 = svc_rbf.predict(np.c_[xx1.ravel(), yy1.ravel()])
Z1 = Z1.reshape(xx1.shape)
plt.contourf(xx1, yy1, Z1, cmap=plt.cm.Paired, alpha=0.8)
plt.scatter(X1[:, 0], X1[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx1.min(), xx1.max())
plt.title('SVC with rbf kernel for third and fourth feature')
plt.show()
```

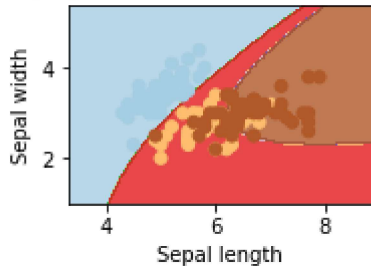
SVC with rbf kernel for third and fourth feature



In [13]:

```
plt.subplot(2, 2, 1)
Z = svc_poly.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx.min(), xx.max())
plt.title('SVC with poly kernel for first and second feature')
plt.show()
```

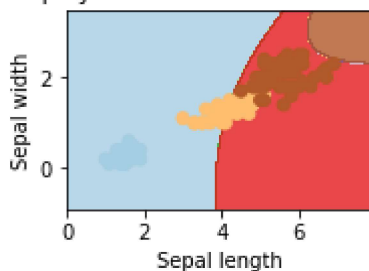
SVC with poly kernel for first and second feature



In [14]:

```
plt.subplot(2, 2, 1)
Z1 = svc_poly.predict(np.c_[xx1.ravel(), yy1.ravel()])
Z1 = Z1.reshape(xx1.shape)
plt.contourf(xx1, yy1, Z1, cmap=plt.cm.Paired, alpha=0.8)
plt.scatter(X1[:, 0], X1[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx1.min(), xx1.max())
plt.title('SVC with poly kernel for third and fourth feature')
plt.show()
```

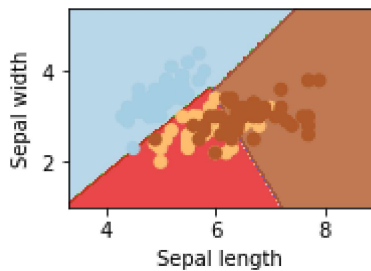
SVC with poly kernel for third and fourth feature



In [15]:

```
plt.subplot(2, 2, 1)
Z = svc_linear.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx.min(), xx.max())
plt.title('SVC with linear kernel for first two feature')
plt.show()
```

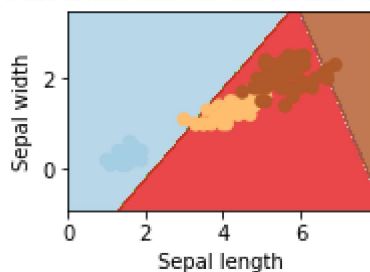
SVC with linear kernel for first two feature



In [16]:

```
plt.subplot(2, 2, 1)
Z1 = svc_linear.predict(np.c_[xx1.ravel(), yy1.ravel()])
Z1 = Z1.reshape(xx1.shape)
plt.contourf(xx1, yy1, Z1, cmap=plt.cm.Paired, alpha=0.8)
plt.scatter(X1[:, 0], X1[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx1.min(), xx1.max())
plt.title('SVC with linear kernel for3rd and 4th feature')
plt.show()
```

SVC with linear kernel for3rd and 4th feature



In [17]:

```
2# get support vectors according to third and forth feature  
svc_rbf.support_vectors_
```

Out[17]:

```
array([[4.9, 3. ],  
       [4.3, 3. ],  
       [5.8, 4. ],  
       [5.7, 4.4],  
       [5.4, 3.4],  
       [5. , 3. ],  
       [5.4, 3.4],  
       [5.5, 3.5],  
       [4.5, 2.3],  
       [4.8, 3. ],  
       [7. , 3.2],  
       [6.4, 3.2],  
       [6.9, 3.1],  
       [6.5, 2.8],  
       [6.3, 3.3],  
       [4.9, 2.4],  
       [6.6, 2.9],  
       [5.2, 2.7],  
       [5. , 2. ],  
       [5.9, 3. ],  
       [6. , 2.2],  
       [6.1, 2.9],  
       [6.7, 3.1],  
       [5.6, 3. ],  
       [5.8, 2.7],  
       [6.2, 2.2],  
       [5.9, 3.2],  
       [6.1, 2.8],  
       [6.3, 2.5],  
       [6.1, 2.8],  
       [6.4, 2.9],  
       [6.6, 3. ],  
       [6.8, 2.8],  
       [6.7, 3. ],  
       [6. , 2.9],  
       [5.8, 2.7],  
       [6. , 2.7],  
       [5.4, 3. ],  
       [6. , 3.4],  
       [6.7, 3.1],  
       [6.3, 2.3],  
       [5.6, 3. ],  
       [6.1, 3. ],  
       [5.8, 2.6],  
       [5. , 2.3],  
       [5.7, 3. ],  
       [5.7, 2.9],  
       [6.2, 2.9],  
       [5.1, 2.5],  
       [5.7, 2.8],  
       [6.3, 3.3],  
       [5.8, 2.7],  
       [6.3, 2.9],  
       [6.5, 3. ]])
```

```
[4.9, 2.5],  
[7.3, 2.9],  
[6.7, 2.5],  
[7.2, 3.6],  
[6.5, 3.2],  
[6.4, 2.7],  
[6.8, 3. ],  
[5.7, 2.5],  
[5.8, 2.8],  
[6.4, 3.2],  
[6.5, 3. ],  
[7.7, 2.6],  
[6. , 2.2],  
[5.6, 2.8],  
[6.3, 2.7],  
[6.2, 2.8],  
[6.1, 3. ],  
[6.4, 2.8],  
[7.9, 3.8],  
[6.4, 2.8],  
[6.3, 2.8],  
[6.1, 2.6],  
[6.3, 3.4],  
[6.4, 3.1],  
[6. , 3. ],  
[6.7, 3.1],  
[5.8, 2.7],  
[6.7, 3. ],  
[6.3, 2.5],  
[6.5, 3. ],  
[6.2, 3.4],  
[5.9, 3. ]])
```

In [18]:

```
svc_poly.support_vectors_
```

Out[18]:

```
array([[5.5, 3.5],
       [4.5, 2.3],
       [7. , 3.2],
       [6.4, 3.2],
       [6.9, 3.1],
       [6.5, 2.8],
       [5.7, 2.8],
       [6.3, 3.3],
       [4.9, 2.4],
       [6.6, 2.9],
       [5.9, 3. ],
       [6.1, 2.9],
       [6.7, 3.1],
       [5.8, 2.7],
       [6.2, 2.2],
       [5.9, 3.2],
       [6.1, 2.8],
       [6.3, 2.5],
       [6.1, 2.8],
       [6.4, 2.9],
       [6.6, 3. ],
       [6.8, 2.8],
       [6.7, 3. ],
       [6. , 2.9],
       [5.7, 2.6],
       [5.8, 2.7],
       [6. , 2.7],
       [6. , 3.4],
       [6.7, 3.1],
       [6.3, 2.3],
       [6.1, 3. ],
       [5.8, 2.6],
       [5.7, 3. ],
       [5.7, 2.9],
       [6.2, 2.9],
       [5.7, 2.8],
       [6.3, 3.3],
       [5.8, 2.7],
       [6.3, 2.9],
       [6.5, 3. ],
       [4.9, 2.5],
       [6.7, 2.5],
       [6.5, 3.2],
       [6.4, 2.7],
       [6.8, 3. ],
       [5.7, 2.5],
       [5.8, 2.8],
       [6.4, 3.2],
       [6.5, 3. ],
       [6. , 2.2],
       [5.6, 2.8],
       [6.3, 2.7],
       [6.7, 3.3],
       [6.2, 2.8],
       [6.1, 3. ]])
```

```
[6.4, 2.8],  
[6.4, 2.8],  
[6.3, 2.8],  
[6.1, 2.6],  
[6.3, 3.4],  
[6.4, 3.1],  
[6. , 3. ],  
[6.7, 3.1],  
[5.8, 2.7],  
[6.7, 3. ],  
[6.3, 2.5],  
[6.5, 3. ],  
[6.2, 3.4],  
[5.9, 3. ]])
```

In [14]:

```
#In accordance to third and forth feature  
svc_linear.support_vectors_
```

Out[14]:

```
array([[4.9, 3. ],  
       [5.7, 3.8],  
       [5.4, 3.4],  
       [5. , 3. ],  
       [5.4, 3.4],  
       [5.5, 3.5],  
       [4.5, 2.3],  
       [4.8, 3. ],  
       [7. , 3.2],  
       [6.4, 3.2],  
       [6.9, 3.1],  
       [6.5, 2.8],  
       [5.7, 2.8],  
       [6.3, 3.3],  
       [4.9, 2.4],  
       [6.6, 2.9],  
       [5.2, 2.7],  
       [5.9, 3. ],  
       [6. , 2.2],  
       [6.1, 2.9],  
       [6.7, 3.1],  
       [5.6, 3. ],  
       [5.8, 2.7],  
       [6.2, 2.2],  
       [5.9, 3.2],  
       [6.1, 2.8],  
       [6.3, 2.5],  
       [6.1, 2.8],  
       [6.4, 2.9],  
       [6.6, 3. ],  
       [6.8, 2.8],  
       [6.7, 3. ],  
       [6. , 2.9],  
       [5.8, 2.7],  
       [6. , 2.7],  
       [5.4, 3. ],  
       [6. , 3.4],  
       [6.7, 3.1],  
       [6.3, 2.3],  
       [5.6, 3. ],  
       [6.1, 3. ],  
       [5.8, 2.6],  
       [5.7, 3. ],  
       [5.7, 2.9],  
       [6.2, 2.9],  
       [5.1, 2.5],  
       [5.7, 2.8],  
       [6.3, 3.3],  
       [5.8, 2.7],  
       [6.3, 2.9],  
       [6.5, 3. ],  
       [4.9, 2.5],  
       [6.7, 2.5],  
       [6.5, 3.2],
```

```
[6.4, 2.7],
[6.8, 3. ],
[5.7, 2.5],
[5.8, 2.8],
[6.4, 3.2],
[6.5, 3. ],
[6. , 2.2],
[5.6, 2.8],
[6.3, 2.7],
[6.7, 3.3],
[6.2, 2.8],
[6.1, 3. ],
[6.4, 2.8],
[6.4, 2.8],
[6.3, 2.8],
[6.1, 2.6],
[6.3, 3.4],
[6.4, 3.1],
[6. , 3. ],
[6.7, 3.1],
[5.8, 2.7],
[6.7, 3.3],
[6.7, 3. ],
[6.3, 2.5],
[6.5, 3. ],
[6.2, 3.4],
[5.9, 3. ]])
```

In [19]:

```
# get indices of support vectors according to 3rd and 4th feature
svc_rbf.support_
```

Out[19]:

```
array([ 1, 13, 14, 15, 20, 25, 31, 36, 41, 45, 50, 51, 52,
        54, 56, 57, 58, 59, 60, 61, 62, 63, 65, 66, 67, 68,
        70, 71, 72, 73, 74, 75, 76, 77, 78, 82, 83, 84, 85,
        86, 87, 88, 91, 92, 93, 95, 96, 97, 98, 99, 100, 101,
       103, 104, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116,
       118, 119, 121, 123, 126, 127, 128, 131, 132, 133, 134, 136, 137,
       138, 140, 142, 145, 146, 147, 148, 149])
```

In [20]:

```
# get indices of support vectors according to 3rd and 4th
svc_linear.support_
```

Out[20]:

```
array([ 1, 18, 20, 25, 31, 36, 41, 45, 50, 51, 52, 54, 55,
        56, 57, 58, 59, 61, 62, 63, 65, 66, 67, 68, 70, 71,
        72, 73, 74, 75, 76, 77, 78, 82, 83, 84, 85, 86, 87,
        88, 91, 92, 95, 96, 97, 98, 99, 100, 101, 103, 104, 106,
       108, 110, 111, 112, 113, 114, 115, 116, 119, 121, 123, 124, 126,
       127, 128, 132, 133, 134, 136, 137, 138, 140, 142, 144, 145, 146,
       147, 148, 149])
```

In [21]:

```
svc_poly.support_
```

Out[21]:

```
array([ 36,  41,  50,  51,  52,  54,  55,  56,  57,  58,  61,  63,  65,
        67,  68,  70,  71,  72,  73,  74,  75,  76,  77,  78,  79,  82,
        83,  85,  86,  87,  91,  92,  95,  96,  97,  99, 100, 101, 103,
       104, 106, 108, 110, 111, 112, 113, 114, 115, 116, 119, 121, 123,
       124, 126, 127, 128, 132, 133, 134, 136, 137, 138, 140, 142, 145,
       146, 147, 148, 149])
```

In [23]:

```
# get number of support vectors for each class
svc_rbf.n_support_
```

Out[23]:

```
array([10, 40, 36])
```

In [24]:

```
svc_poly.n_support_
```

Out[24]:

```
array([ 2, 34, 33])
```

In [25]:

```
svc_linear.n_support_
```

Out[25]:

```
array([ 8, 39, 34])
```

Use the feature column 3 and 4 for the classification and compare your result

Thanks:

1. <https://www.analyticsvidhya.com/blog/2017/09/understaing-support-vector-machine-example-code/>
(<https://www.analyticsvidhya.com/blog/2017/09/understaing-support-vector-machine-example-code/>)